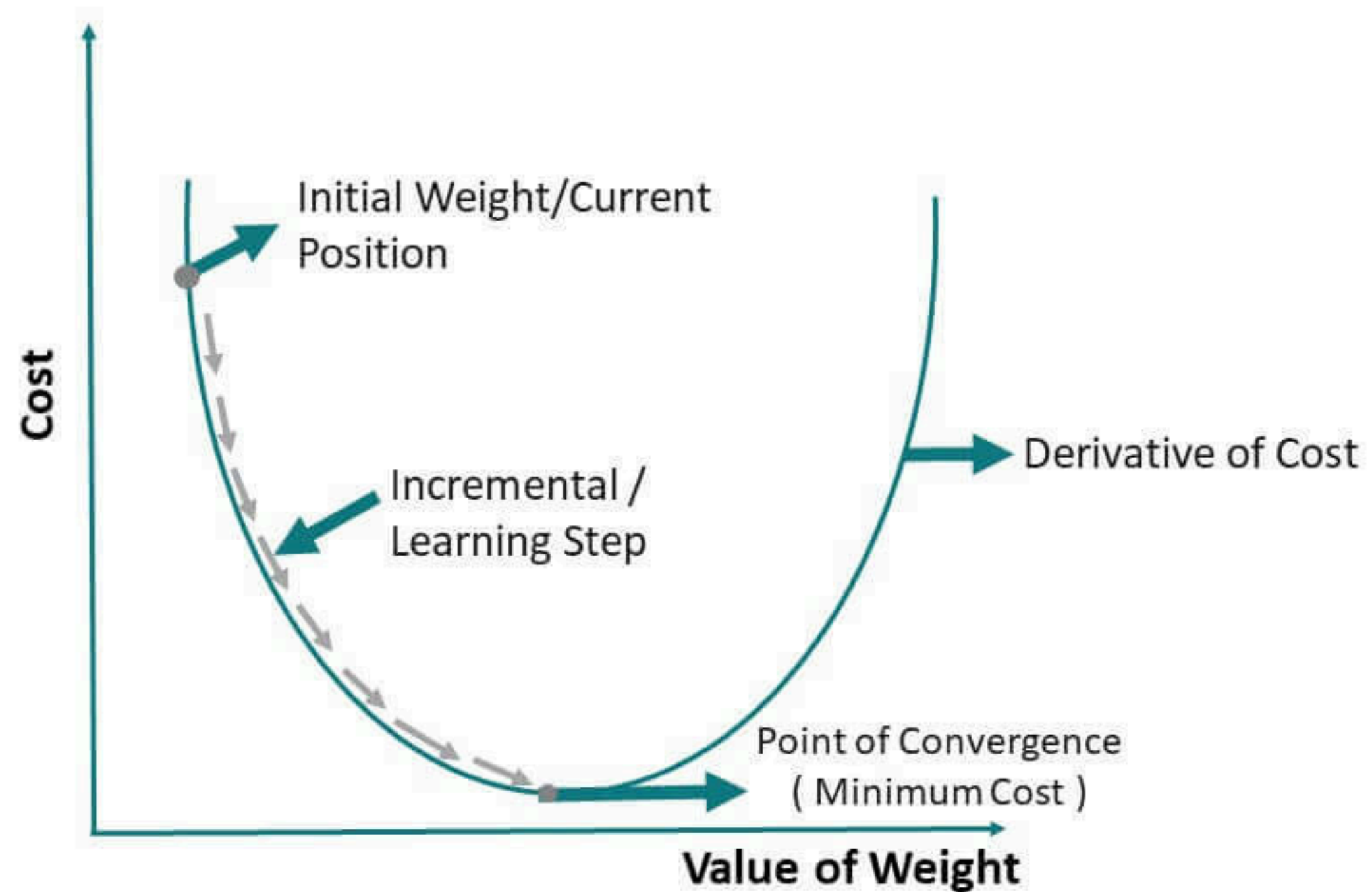
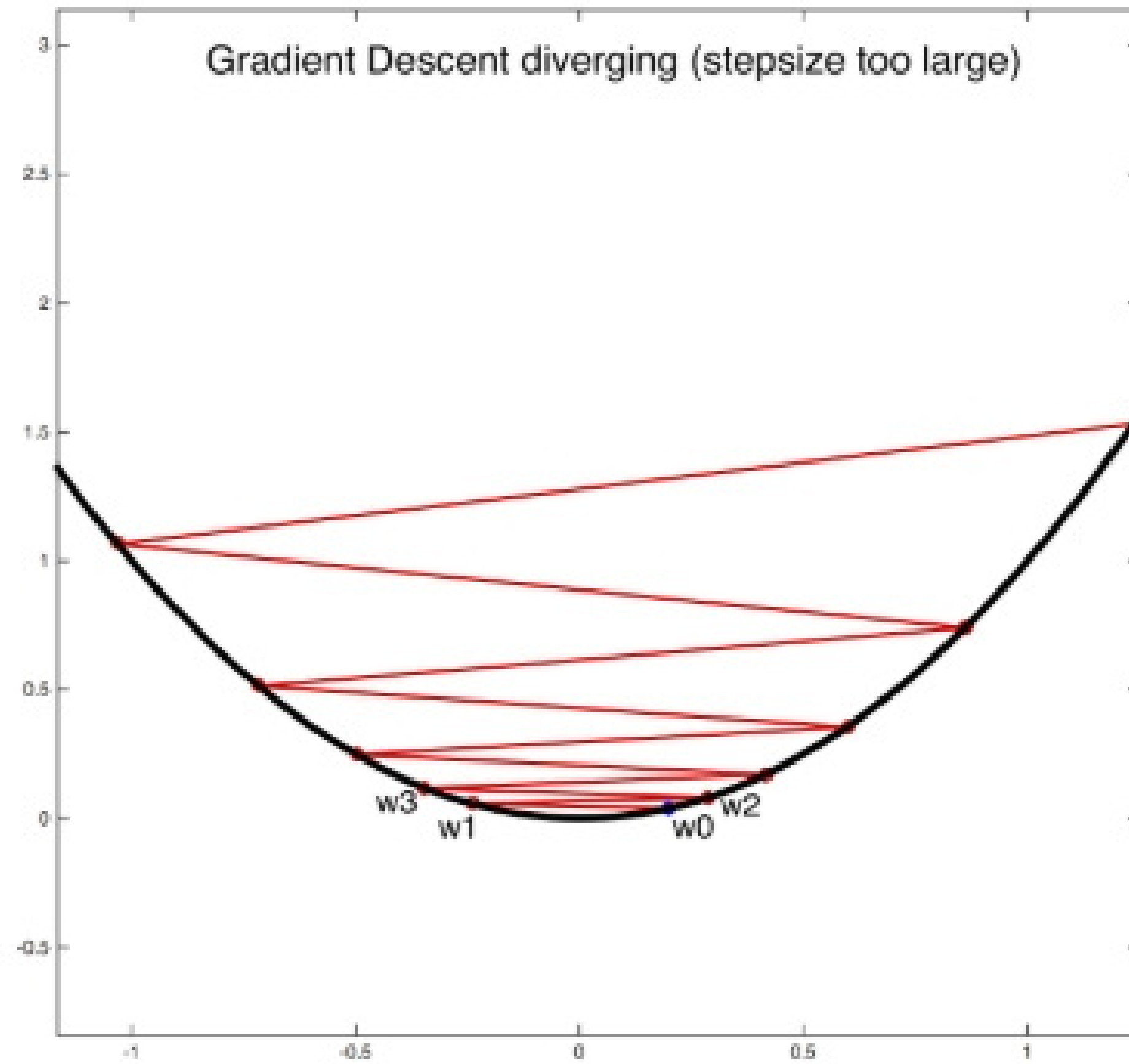


Gradient Descent

Gradient Descent of Machine Learning





1. Batch Gradient Descent

"The Deep Thinker"

In this method, you process the **entire dataset** (all 1,000 points) to calculate one single average gradient.

- **The Update:** You update w and b only **once** per epoch.
- **The Path:** Extremely smooth and direct toward the minimum.
- **Best for:** Small datasets that fit easily into your computer's memory.
- **The Downside:** It is very slow. If you have 1 million points, you wait a long time for just one tiny update.

2. Stochastic Gradient Descent (SGD)

"The Quick Reactor"

You treat every single data point like a mini-lesson. You look at Point #1 and immediately change your w and b .

- **The Update:** You update w and b **1,000 times** if you have 1,000 points.
- **The Path:** Very "noisy" and zig-zaggy. Because it reacts to every single point (even outliers), it jumps around a lot.
- **Best for:** Very large datasets or models that need to "jump" out of bad local minima.
- **The Downside:** It never truly "settles" at the exact minimum because it's always twitching based on the last point it saw.

3. Mini-Batch Gradient Descent

"The Modern Standard"

You split your 1,000 points into small "batches" (e.g., 32 points each). You average the error for those 32 points and then update.

- **The Update:** You update w and b after every batch. (e.g., 1,000 points / 32 = **31 updates** per epoch).
- **The Path:** A perfect balance. It's faster than Batch and smoother than SGD.
- **Best for:** Almost everything. This is what modern AI (like ChatGPT or Image generators) uses.
- **The Logic:** It uses "Vectorization," which allows your computer hardware (GPU) to calculate those 32 points at the exact same time.

The Final Rule

Regardless of the technique:

1. We calculate **Gradients** (derivatives) to know the direction.
2. We use a **Learning Rate** (α) to control the step size.
3. We repeat for **Many Epochs** until the **Overall Loss** stops dropping and the derivative is close to zero.

Step 0: Dataset and Initial Values

x	y
1	2
2	3
3	4
4	5
5	6

- Linear regression: $y = wx + b$
- Initial weights: $w = 0, b = 0$
- Learning rate: $\alpha = 0.1$
- Loss: MSE

Batch gradient formulas:

$$\frac{\partial L}{\partial w} = -\frac{2}{n} \sum_{i=1}^n x_i (y_i - (wx_i + b))$$

$$\frac{\partial L}{\partial b} = -\frac{2}{n} \sum_{i=1}^n (y_i - (wx_i + b))$$

$$w := w - \alpha \frac{\partial L}{\partial w}, \quad b := b - \alpha \frac{\partial L}{\partial b}$$

Step 1: Compute Predictions and Errors (Initial Weights)

x	y	Prediction $\hat{y} = wx + b$	Error $e = y - \hat{y}$
1	2	0	2
2	3	0	3
3	4	0	4
4	5	0	5
5	6	0	6

Step 2: Compute Gradients (Batch GD)

$$\begin{aligned}\frac{\partial L}{\partial w} &= -\frac{2}{5} \sum x_i (y_i - \hat{y}_i) = -\frac{2}{5} (1 * 2 + 2 * 3 + 3 * 4 + 4 * 5 + 5 * 6) \\ &= -\frac{2}{5} (2 + 6 + 12 + 20 + 30) = -\frac{2}{5} * 70 = -28\end{aligned}$$

$$\frac{\partial L}{\partial b} = -\frac{2}{5} \sum (y_i - \hat{y}_i) = -\frac{2}{5} (2 + 3 + 4 + 5 + 6) = -\frac{2}{5} * 20 = -8$$

Step 3: Update Weights

$$w_{\text{new}} = w - \alpha \frac{\partial L}{\partial w} = 0 - 0.1 * (-28) = 2.8$$

$$b_{\text{new}} = b - \alpha \frac{\partial L}{\partial b} = 0 - 0.1 * (-8) = 0.8$$

✓ Updated weights after 1 epoch: $w = 2.8, b = 0.8$

Step 4: Compute Predictions with Updated Weights

x	y	Prediction $\hat{y} = wx + b$	Error $e = y - \hat{y}$	Squared Error
1	2	$2.8 \cdot 1 + 0.8 = 3.6$	$2 - 3.6 = -1.6$	2.56
2	3	$2.8 \cdot 2 + 0.8 = 6.4$	$3 - 6.4 = -3.4$	11.56
3	4	$2.8 \cdot 3 + 0.8 = 9.2$	$4 - 9.2 = -5.2$	27.04
4	5	$2.8 \cdot 4 + 0.8 = 12.0$	$5 - 12.0 = -7.0$	49.00
5	6	$2.8 \cdot 5 + 0.8 = 14.8$	$6 - 14.8 = -8.8$	77.44

Step 5: Compute Overall MSE Loss

$$\text{MSE} = \frac{1}{5} \sum (y_i - \hat{y}_i)^2 = \frac{1}{5} (2.56 + 11.56 + 27.04 + 49 + 77.44) = \frac{167.6}{5} \approx 33.52$$

✓ Loss after 1 epoch: 33.52

- Initial loss (with $w=0$, $b=0$) would be:

$$L = \frac{1}{5} \sum (y_i - 0)^2 = \frac{1}{5} (4 + 9 + 16 + 25 + 36) = \frac{90}{5} = 18$$

Step 6: Repeat for Next Epoch (Convergence)

- Recompute gradients using updated weights $w = 2.8, b = 0.8$
- Update weights again: $w_{\text{new}} = w - \alpha \frac{\partial L}{\partial w}, b_{\text{new}} = b - \alpha \frac{\partial L}{\partial b}$
- Compute new MSE
- Repeat until:
 1. Gradients ≈ 0
 2. Loss stops decreasing $\rightarrow$ minimum reached

Step 7: Learning Rate Effect

Learning Rate	Effect
Too large	Overshoot → loss can increase or oscillate
Too small	Slow convergence → need many epochs
Good	Smooth convergence → gradually reaches minimum MSE

✓ Key Notes:

- **Batch Gradient Descent:** compute gradient for **all data points first**, then update weights **once per epoch**.
- **Minimum loss:** reached when gradients ≈ 0 and loss stops decreasing.

Step 0: Setup for Mini-Batch GD

- Dataset:

x	y
1	2
2	3
3	4
4	5
5	6

- Initial weights: $w = 0, b = 0$
- Learning rate: $\alpha = 0.1$
- Mini-batch size: 2 (except last batch may have fewer points)
- Loss: MSE

Mini-batch gradient formulas (for a batch of size m):

$$\frac{\partial L}{\partial w} = -\frac{2}{m} \sum_{i=1}^m x_i (y_i - (wx_i + b))$$

$$\frac{\partial L}{\partial b} = -\frac{2}{m} \sum_{i=1}^m (y_i - (wx_i + b))$$

Update weights after each mini-batch.

Step 1: Split into Mini-Batches

- Batch 1: $(x=1, y=2), (x=2, y=3)$
- Batch 2: $(x=3, y=4), (x=4, y=5)$
- Batch 3: $(x=5, y=6)$

Step 2: Mini-Batch 1 Update

Predictions (initial weights $w=0$, $b=0$)

x	y	Prediction	Error
1	2	0	2
2	3	0	3

- Batch size $m = 2$
- Gradients:

$$\frac{\partial L}{\partial w} = -\frac{2}{2}(1 * 2 + 2 * 3) = -1 * (2 + 6) = -8$$

$$\frac{\partial L}{\partial b} = -\frac{2}{2}(2 + 3) = -5$$

- Update weights:

$$w = 0 - 0.1 * (-8) = 0.8$$

$$b = 0 - 0.1 * (-5) = 0.5$$

✓ Updated weights after batch 1: $w = 0.8, b = 0.5$

Step 3: Mini-Batch 2 Update

Predictions with $w=0.8$, $b=0.5$

x	y	Prediction	Error
3	4	$0.8*3+0.5=2.9$	$4-2.9=1.1$
4	5	$0.8*4+0.5=3.7$	$5-3.7=1.3$

- Gradients:

$$\frac{\partial L}{\partial w} = -\frac{2}{2}(3 * 1.1 + 4 * 1.3) = -1 * (3.3 + 5.2) = -8.5$$

$$\frac{\partial L}{\partial b} = -\frac{2}{2}(1.1 + 1.3) = -2.4$$

- Update weights:

$$w = 0.8 - 0.1 * (-8.5) = 0.8 + 0.85 = 1.65$$

$$b = 0.5 - 0.1 * (-2.4) = 0.5 + 0.24 = 0.74$$

✓ Updated weights after batch 2: $w = 1.65$, $b = 0.74$

Step 4: Mini-Batch 3 Update

Predictions with $w=1.65$, $b=0.74$

x	y	Prediction	Error
5	6	$1.65 * 5 + 0.74 = 8.99$	$6 - 8.99 = -2.99$

- Batch size $m=1$
- Gradients:

$$\frac{\partial L}{\partial w} = -2 * 5 * (-2.99) = 29.9$$

$$\frac{\partial L}{\partial b} = -2 * (-2.99) = 5.98$$

- Update weights:

$$w = 1.65 - 0.1 * 29.9 = 1.65 - 2.99 = -1.34$$

$$b = 0.74 - 0.1 * 5.98 = 0.74 - 0.598 = 0.142$$

✓ Updated weights after batch 3 (end of epoch): $w = -1.34$, $b = 0.142$

Step 5: Compute Overall MSE after 1 Epoch

- Predictions with final weights:

$$\hat{y}_i = [-1.34 * 1 + 0.142, -1.34 * 2 + 0.142, -1.34 * 3 + 0.142, -1.34 * 4 + 0.142, -1.34 * 5 + 0.142]$$

$$\hat{y}_i \approx [-1.198, -2.538, -3.878, -5.218, -6.558]$$

- Errors $y - \hat{y} = [2 + 1.198, 3 + 2.538, 4 + 3.878, 5 + 5.218, 6 + 6.558] \approx [3.198, 5.538, 7.878, 10.218, 12.558]$
- Squared errors $[10.23, 30.68, 62.05, 104.41, 157.73]$
- $MSE = \frac{1}{5} \sum = \frac{364.1}{5} \approx 72.82$

Loss increased because learning rate 0.1 is too high for mini-batches → shows sensitivity of mini-batch GD.

✓ Summary of Mini-Batch GD

Batch	Updated w	Updated b	Notes
1	0.8	0.5	first 2 points
2	1.65	0.74	next 2 points
3	-1.34	0.142	last point

- Mini-Batch GD: update after each small batch, not after each sample (SGD) or full dataset (Batch GD)
- Can converge faster than Batch GD, but sensitive to learning rate.

$$L = e^2 = (y - \hat{y})^2$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$L = (y - (wx + b))^2 = e^2$$

$$\frac{\partial L}{\partial w} = 2e \cdot \frac{\partial e}{\partial w}$$

$$\frac{\partial e}{\partial w} = \frac{\partial}{\partial w}(y - wx - b) = -x$$

$$\frac{\partial L}{\partial w} = 2e \cdot (-x) = -2xe$$

$$\frac{\partial L}{\partial w} = -2xe$$

$$\frac{\partial L}{\partial b} = 2e \cdot \frac{\partial e}{\partial b}$$

$$\frac{\partial e}{\partial b} = \frac{\partial}{\partial b}(y - wx - b) = -1$$

$$\frac{\partial L}{\partial b} = 2e \cdot (-1) = -2e$$

$$\frac{\partial L}{\partial b} = -2e$$

$$\hat{y} = wx + b$$

$$\text{error } e = y - \hat{y}$$

$$\frac{\partial L}{\partial w} = -2xe, \quad \frac{\partial L}{\partial b} = -2e$$

$$w := w - \alpha \frac{\partial L}{\partial w}, \quad b := b - \alpha \frac{\partial L}{\partial b}$$

Step 1: SGD Updates for Each Data Point

Data point 1: $x=1, y=2$

- Prediction: $\hat{y} = 0 * 1 + 0 = 0$ ✓
- Error: $e = 2 - 0 = 2$ ✓
- Gradients:

$$\frac{\partial L}{\partial w} = -2 * 1 * 2 = -4$$

$$\frac{\partial L}{\partial b} = -2 * 2 = -4$$

- Update:

$$w = 0 - 0.1 * (-4) = 0.4$$

$$b = 0 - 0.1 * (-4) = 0.4$$

✓ New weights: $w = 0.4, b = 0.4$

Data point 2: $x=2, y=3$

- Prediction: $\hat{y} = 0.4 * 2 + 0.4 = 1.2$
- Error: $e = 3 - 1.2 = 1.8$
- Gradients:

$$\frac{\partial L}{\partial w} = -2 * 2 * 1.8 = -7.2$$

$$\frac{\partial L}{\partial b} = -2 * 1.8 = -3.6$$

- Update:

$$w = 0.4 - 0.1 * (-7.2) = 1.12$$

$$b = 0.4 - 0.1 * (-3.6) = 0.76$$

✓ New weights: $w = 1.12, b = 0.76$

Data point 3: $x=3, y=4$

- Prediction: $\hat{y} = 1.12 * 3 + 0.76 = 4.12$
- Error: $e = 4 - 4.12 = -0.12$
- Gradients:

$$\frac{\partial L}{\partial w} = -2 * 3 * (-0.12) = 0.72$$

$$\frac{\partial L}{\partial b} = -2 * (-0.12) = 0.24$$

- Update:

$$w = 1.12 - 0.1 * 0.72 = 1.048$$

$$b = 0.76 - 0.1 * 0.24 = 0.736$$

✓ New weights: $w = 1.048, b = 0.736$

Data point 4: $x=4, y=5$

- Prediction: $\hat{y} = 1.048 * 4 + 0.736 = 5.928$
- Error: $e = 5 - 5.928 = -0.928$
- Gradients:

$$\frac{\partial L}{\partial w} = -2 * 4 * (-0.928) = 7.424$$

$$\frac{\partial L}{\partial b} = -2 * (-0.928) = 1.856$$

- Update:

$$w = 1.048 - 0.1 * 7.424 = 0.3056$$

$$b = 0.736 - 0.1 * 1.856 = 0.5504$$

✓ New weights: $w = 0.3056, b = 0.5504$

Data point 5: $x=5, y=6$

- Prediction: $\hat{y} = 0.3056 * 5 + 0.5504 = 2.0784$
- Error: $e = 6 - 2.0784 = 3.9216$
- Gradients:

$$\frac{\partial L}{\partial w} = -2 * 5 * 3.9216 = -39.216$$

$$\frac{\partial L}{\partial b} = -2 * 3.9216 = -7.8432$$

- Update:

$$w = 0.3056 - 0.1 * (-39.216) = 4.2262$$

$$b = 0.5504 - 0.1 * (-7.8432) = 1.3347$$

✓ End of Epoch 1 weights: $w \approx 4.226, b \approx 1.335$

- Loss (MSE):

$$L = \frac{1}{5} \sum (y_i - \hat{y}_i)^2$$

$$L \approx \frac{1}{5} [(2 - 5.561)^2 + (3 - 9.787)^2 + \dots + (6 - 22.465)^2]$$

$$L \approx 130.7$$

Loss is still very high because LR=0.1 is too big and weights jumped too far.

Step 3: How to Know Minimum Loss?

1. Minimum loss is reached when gradients are very small (close to 0):

$$\frac{\partial L}{\partial w} \approx 0, \quad \frac{\partial L}{\partial b} \approx 0$$

2. Also, loss stops decreasing significantly over epochs → convergence.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$